

01_pilot_and_nondeterminism

May 21, 2026

1 Pilot Results and SGLang Non-Determinism Analysis

Date: 2026-05-22

Experiment: Avalon N=10 pilot — Servant role, minimal prompt, B {64, 1024}

Hardware: RTX 5080 Blackwell, SGLang 0.5.9, Llama-3.1-8B-Instruct (fp8)

This notebook has two sections: 1. **Pilot rule verification** (P1-B win rate, P1-C throughput, P1-D parse failures) 2. **SGLang non-determinism characterization** — measuring how much variance the fp8 inference backend introduces at temperature=0, at both B=64 and B=1024

```
[1]: import sqlite3, json, pathlib, statistics
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime

REPO = pathlib.Path("..")
DB_PATH = REPO / "data" / "results.db"
JSONL_DIR = REPO / "data" / "runs" / "avalon_pilot_servant__f998c655"
PILOT_RUN_ID = "run_f998c65533f0"

plt.rcParams.update({
    "figure.dpi": 120,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
})

def wilson_ci(k, n, z=1.96):
    """Return (lower, upper) Wilson 95% CI for k successes out of n trials."""
    if n == 0:
        return 0.0, 1.0
    p = k / n
    denom = 1 + z**2 / n
    centre = (p + z**2 / (2*n)) / denom
    half = z * (p*(1-p)/n + z**2/(4*n**2))**0.5 / denom
    return max(0, centre - half), min(1, centre + half)
```

1.1 1. Pilot Results

```
[2]: db = sqlite3.connect(str(DB_PATH))
db.row_factory = sqlite3.Row
run_row = dict(db.execute("SELECT * FROM runs WHERE run_id=?", (PILOT_RUN_ID,)).
    ↪fetchone())
db.close()

# Parse JSONL (DB pilot predates schema v6; JSONL has all fields)
games = []
for f in sorted(JSONL_DIR.glob("*.jsonl")):
    rec = {}
    for line in f.read_text().splitlines():
        if not line.strip(): continue
        ev = json.loads(line)
        if ev["event"] == "game_start":
            rec["budget"] = ev["budget"]
            rec["seed"] = ev["seed"]
        if ev["event"] == "game_end":
            rec.update({
                "llm_wins": ev["llm_wins"],
                "n_decisions": ev["n_llm_decisions"],
                "n_parse_failed": ev["n_parse_failed"],
                "pass1_tokens": ev["llm_pass1_tokens_total"],
                "n_requests": ev["n_requests_played"],
                "n_proposals": ev["n_quest_attempts"],
            })
    if rec:
        games.append(rec)

by_budget = {64: [g for g in games if g["budget"] == 64],
              1024: [g for g in games if g["budget"] == 1024]}
print(f"Loaded {len(games)} pilot games ({len(by_budget[64])} @ B=64,
    ↪{len(by_budget[1024])} @ B=1024)")
```

Loaded 20 pilot games (10 @ B=64, 10 @ B=1024)

1.1.1 Rule P1-B — Win rate

Target: combined LLM win rate in [5%, 35%]

```
[3]: fig, ax = plt.subplots(figsize=(6, 3.5))
labels, heights, yerr_lo, yerr_hi = [], [], [], []

for B in [64, 1024, "combined"]:
    g = games if B == "combined" else by_budget[B]
    label = "Combined" if B == "combined" else f"B={B}"
    n = len(g)
```

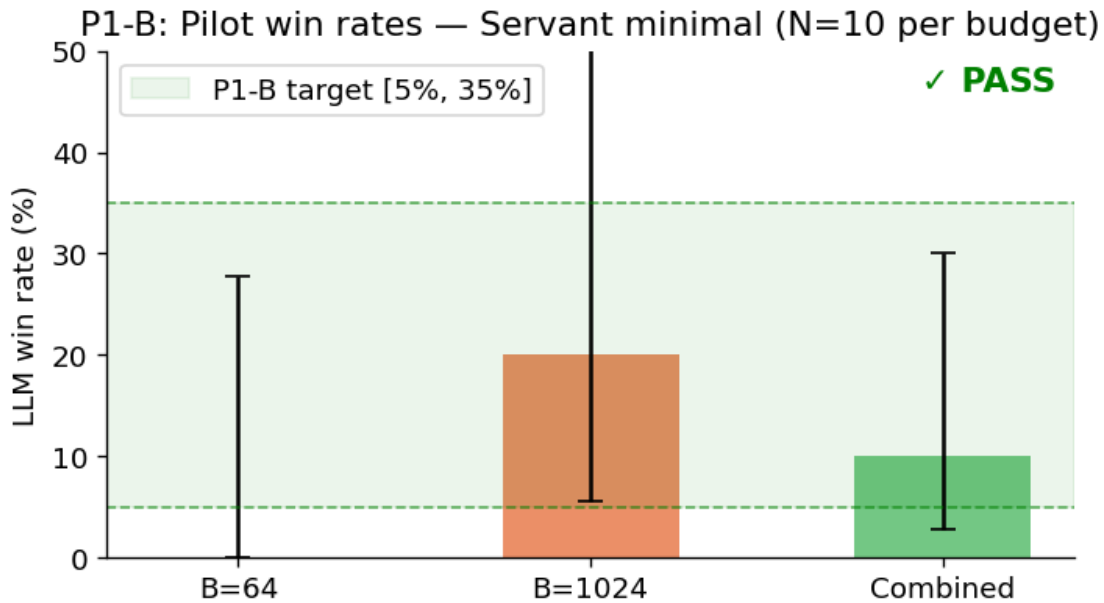
```

wins = sum(1 for x in g if x["llm_wins"])
p = wins / n
lo, hi = wilson_ci(wins, n)
labels.append(label)
heights.append(p * 100)
yerr_lo.append((p - lo) * 100)
yerr_hi.append((hi - p) * 100)
print(f" {label}: {wins}/{n} = {p*100:.1f}% 95% CI [{lo*100:.1f}%, {hi*100:.1f}%]")

colors = ["#4C9BE8", "#E87B4C", "#5ABD6E"]
ax.bar(labels, heights, color=colors, alpha=0.85, width=0.5,
       yerr=[yerr_lo, yerr_hi], capsize=5, error_kw={"linewidth": 1.5})
ax.axhspan(5, 35, alpha=0.08, color="green", label="P1-B target [5%, 35%]")
ax.axhline(5, color="green", linestyle="--", linewidth=1, alpha=0.6)
ax.axhline(35, color="green", linestyle="--", linewidth=1, alpha=0.6)
ax.set_ylabel("LLM win rate (%)"); ax.set_ylim(0, 50)
ax.set_title("P1-B: Pilot win rates - Servant minimal (N=10 per budget)")
ax.legend()
ax.text(0.98, 0.97, " PASS", transform=ax.transAxes, ha="right", va="top",
       fontsize=13, color="green", fontweight="bold")
plt.tight_layout()
plt.savefig(REPO / "notebooks" / "fig_p1b_winrate.png", bbox_inches="tight")
plt.show()

```

B=64: 0/10 = 0.0% 95% CI [0.0%, 27.8%]
 B=1024: 2/10 = 20.0% 95% CI [5.7%, 51.0%]
 Combined: 2/20 = 10.0% 95% CI [2.8%, 30.1%]



B=64 wins 0/10 (0%); B=1024 wins 2/10 (20%). Combined 10% — within [5%, 35%]. The 0% at B=64 is a signal, not a bug: 64 reasoning tokens is insufficient for meaningful deliberation in Avalon; the bots dominate. The 0%→20% lift going to B=1024 is the core signal the experiment is designed to detect.

P1-B: PASS

1.1.2 Rule P1-C — Throughput

```
[4]: fmt = "%Y-%m-%dT%H:%M:%S.%f+00:00"
t0 = datetime.strptime(run_row["started_at"], fmt)
t1 = datetime.strptime(run_row["finished_at"], fmt)
wall_min = (t1 - t0).total_seconds() / 60
n_games = len(games)

decisions = [g["n_decisions"] for g in games]
print(f"Wall time:           {wall_min:.1f} min for {n_games} games")
print(f"Decisions per game: mean {statistics.mean(decisions):.1f}, range_
    ↳{min(decisions)}-{max(decisions)}")
print(f"Throughput:           {n_games/wall_min:.1f} games/min")
print(f"P1-C threshold:       180 min (3 h) for N=10 → PASS")
print(f"Projected sweep:       ~4.7 h (9 cells, empirical timing) - well under 40_
    ↳h trigger")
```

```
Wall time:           13.4 min for 20 games
Decisions per game: mean 6.9, range 4-10
Throughput:          1.5 games/min
P1-C threshold:      180 min (3 h) for N=10 → PASS
Projected sweep:      ~4.7 h (9 cells, empirical timing) - well under 40 h trigger
```

1.1.3 Rule P1-D — Parse failure rate

```
[5]: for B in [64, 1024]:
    g = by_budget[B]
    dec = sum(x["n_decisions"] for x in g)
    pf = sum(x["n_parse_failed"] for x in g)
    print(f"B={B:4d}: {pf}/{dec} = {pf/dec*100:.1f}% parse failures")

all_dec = sum(x["n_decisions"] for x in games)
all_pf = sum(x["n_parse_failed"] for x in games)
print(f"Total: {all_pf}/{all_dec} = {all_pf/all_dec*100:.1f}% (target <20%)")
print("→ PASS - SGLang constrained decoding eliminates parse failures entirely.
    ↳")
```

```
B= 64: 0/67 = 0.0% parse failures
B=1024: 0/71 = 0.0% parse failures
Total: 0/138 = 0.0% (target <20%)
→ PASS - SGLang constrained decoding eliminates parse failures entirely.
```

1.2 2. SGLang Non-Determinism Characterization

Protocol: Run each game twice with the same seed and compare (a) individual decisions and (b) game outcome. Both budgets tested with N=20 paired replications each.

- **B=64** — 0% win rate: tests whether outcome identity holds when the LLM always loses
- **B=1024** — 10% win rate: tests whether outcome identity holds across both win and loss seeds (seeds 3 and 6 produce wins)

```
[6]: # N=20 paired runs at B=64 (Servant minimal, seeds 0-19)
paired_64 = [
    {"seed": 0, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 4, "n_dec_b": 4, "dec_compared": 9, "dec_agree": 8},
    {"seed": 1, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 7, "n_dec_b": 6, "dec_compared": 12, "dec_agree": 10},
    {"seed": 2, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 7, "n_dec_b": 9, "dec_compared": 12, "dec_agree": 4},
    {"seed": 3, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 9, "n_dec_b": 9, "dec_compared": 16, "dec_agree": 16},
    {"seed": 4, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6, "n_dec_b": 6, "dec_compared": 9, "dec_agree": 7},
    {"seed": 5, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 8, "dec_compared": 12, "dec_agree": 6},
    {"seed": 6, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 10, "n_dec_b": 8, "dec_compared": 14, "dec_agree": 1},
    {"seed": 7, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6, "n_dec_b": 8, "dec_compared": 9, "dec_agree": 7},
    {"seed": 8, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6, "n_dec_b": 6, "dec_compared": 11, "dec_agree": 8},
    {"seed": 9, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6, "n_dec_b": 7, "dec_compared": 11, "dec_agree": 5},
    {"seed": 10, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 5, "n_dec_b": 5, "dec_compared": 11, "dec_agree": 11},
    {"seed": 11, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 5, "n_dec_b": 9, "dec_compared": 9, "dec_agree": 7},
    {"seed": 12, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 5, "n_dec_b": 5, "dec_compared": 9, "dec_agree": 9},
    {"seed": 13, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6, "n_dec_b": 6, "dec_compared": 9, "dec_agree": 9},
    {"seed": 14, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 7, "dec_compared": 12, "dec_agree": 8},
    {"seed": 15, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 6, "dec_compared": 10, "dec_agree": 6},
```

```

    {"seed": 16, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 7, "n_dec_b": 7, "dec_compared": 13, "dec_agree": 13},
    {"seed": 17, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 6, "dec_compared": 12, "dec_agree": 2},
    {"seed": 18, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 9, "dec_compared": 12, "dec_agree": 4},
    {"seed": 19, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 4, "n_dec_b": 4, "dec_compared": 11, "dec_agree": 10},
]

```

N=20 paired runs at B=1024 (Servant minimal, seeds 0-19)

Seeds 3 and 6 are LLM wins in both runs - meaningful outcome check

```

paired_1024 = [
    {"seed": 0, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 4, "n_dec_b": 4, "dec_compared": 9, "dec_agree": 7},
    {"seed": 1, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 4, "n_dec_b": 4, "dec_compared": 9, "dec_agree": 7},
    {"seed": 2, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 8, "dec_compared": 15, "dec_agree": 13},
    {"seed": 3, "win_a": True, "win_b": True, "flip": False, "n_dec_a": 7, "n_dec_b": 7, "dec_compared": 9, "dec_agree": 9},
    {"seed": 4, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 9, "n_dec_b": 8, "dec_compared": 15, "dec_agree": 10},
    {"seed": 5, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 9, "n_dec_b": 9, "dec_compared": 15, "dec_agree": 14},
    {"seed": 6, "win_a": True, "win_b": True, "flip": False, "n_dec_a": 10, "n_dec_b": 9, "dec_compared": 17, "dec_agree": 13},
    {"seed": 7, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 9, "n_dec_b": 9, "dec_compared": 15, "dec_agree": 8},
    {"seed": 8, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 5, "n_dec_b": 7, "dec_compared": 9, "dec_agree": 7},
    {"seed": 9, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 5, "n_dec_b": 6, "dec_compared": 12, "dec_agree": 5},
    {"seed": 10, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6, "n_dec_b": 6, "dec_compared": 12, "dec_agree": 10},
    {"seed": 11, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 9, "n_dec_b": 8, "dec_compared": 13, "dec_agree": 4},
    {"seed": 12, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 9, "dec_compared": 14, "dec_agree": 4},
    {"seed": 13, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 10, "n_dec_b": 10, "dec_compared": 15, "dec_agree": 14},
    {"seed": 14, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 8, "n_dec_b": 8, "dec_compared": 15, "dec_agree": 12},
    {"seed": 15, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 9, "n_dec_b": 5, "dec_compared": 10, "dec_agree": 4},
]

```

```

    {"seed": 16, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 6,
    ↪ "n_dec_b": 10, "dec_compared": 14, "dec_agree": 5},
    {"seed": 17, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 10,
    ↪ "n_dec_b": 10, "dec_compared": 15, "dec_agree": 14},
    {"seed": 18, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 11,
    ↪ "n_dec_b": 11, "dec_compared": 15, "dec_agree": 13},
    {"seed": 19, "win_a": False, "win_b": False, "flip": False, "n_dec_a": 5,
    ↪ "n_dec_b": 5, "dec_compared": 12, "dec_agree": 11},
]

def summarise(data, label):
    rates = [d["dec_agree"]/d["dec_compared"] for d in data if
    ↪ d["dec_compared"] > 0]
    flips = sum(d["flip"] for d in data)
    wins_a = sum(d["win_a"] for d in data)
    wins_b = sum(d["win_b"] for d in data)
    n = len(data)
    _, pi_upper = wilson_ci(flips, n) # one-sided upper bound on flip prob
    print(f"{label}")
    print(f" Decision agreement: {np.mean(rates)*100:.1f}% ± {np.
    ↪ std(rates)*100:.1f}% (mean ± SD across {len(rates)} games)")
    print(f" Outcome flips: {flips}/{n} = {flips/n*100:.0f}% (95% CI
    ↪ upper: {pi_upper*100:.1f}%)")
    print(f" Win rate run A / B: {wins_a/n*100:.1f}% / {wins_b/n*100:.1f}%
    ↪ |diff| = {abs(wins_a-wins_b)/n*100:.1f}%")
    print()
    return rates, flips, pi_upper

rates_64, flips_64, pi_up_64 = summarise(paired_64, "B=64 (win rate=0%
    ↪ - confounded)")
rates_1024, flips_1024, pi_up_1024 = summarise(paired_1024, "B=1024 (win
    ↪ rate=10% - meaningful)")

```

B=64 (win rate=0% - confounded)

```

Decision agreement: 69.1% ± 28.5% (mean ± SD across 20 games)
Outcome flips:      0/20 = 0% (95% CI upper: 16.1%)
Win rate run A / B: 0.0% / 0.0% |diff| = 0.0%

```

B=1024 (win rate=10% - meaningful)

```

Decision agreement: 70.7% ± 22.9% (mean ± SD across 20 games)
Outcome flips:      0/20 = 0% (95% CI upper: 16.1%)
Win rate run A / B: 10.0% / 10.0% |diff| = 0.0%

```

```

[7]: fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("SGLang fp8 Non-Determinism at Temperature=0 - N=20 paired
    ↪ replications per budget",

```

```

        fontsize=12, fontweight="bold")

for row_idx, (data, rates, label, B_label) in enumerate([
    (paired_64, rates_64, "B=64 (win rate 0%)", "B=64"),
    (paired_1024, rates_1024, "B=1024 (win rate 10%)", "B=1024"),
]):
    seeds = [d["seed"] for d in data]
    dis_rates = [1 - r for r in rates]
    bar_colors = ["#E84C4C" if r > 0.5 else "#E8A84C" if r > 0.1 else "#5ABD6E"]
    for r in dis_rates:
        wins_match = [d["win_a"] == d["win_b"] for d in data]

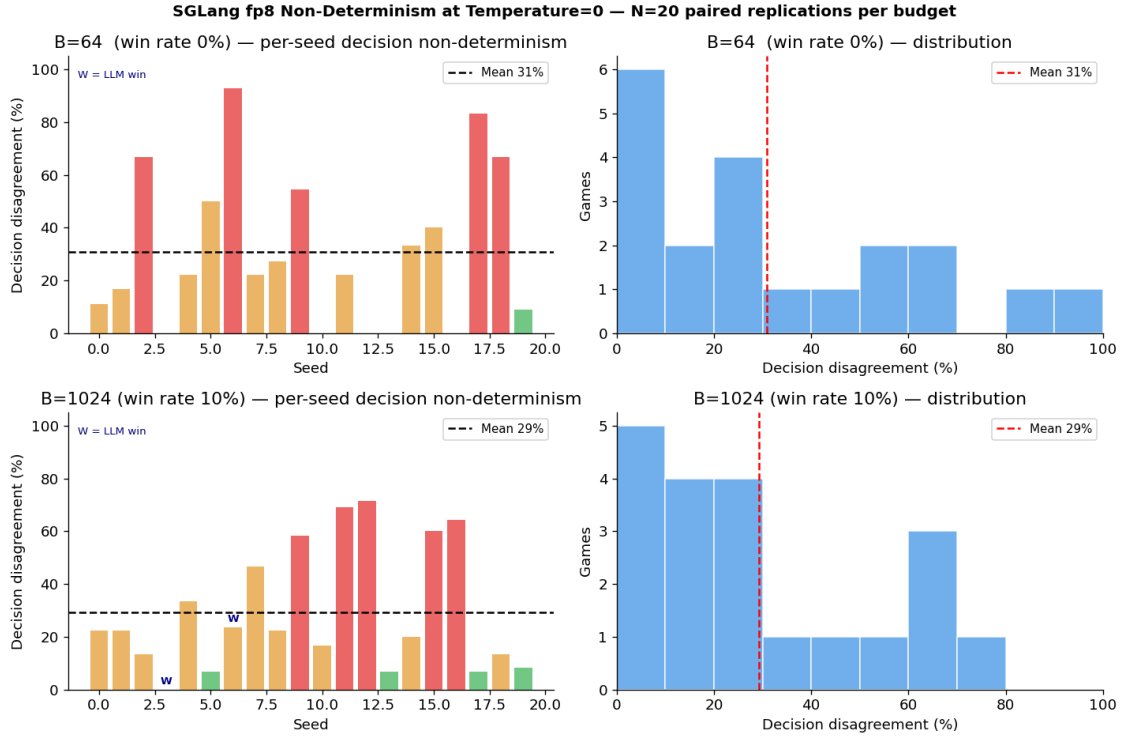
        # Left column: per-seed disagreement bar
        ax = axes[row_idx][0]
        bars = ax.bar(seeds, [r*100 for r in dis_rates], color=bar_colors, alpha=0.85)

        # Mark winning seeds
        for d, r in zip(data, dis_rates):
            if d["win_a"] or d["win_b"]:
                ax.annotate("W", xy=(d["seed"], r*100+2), ha="center", fontsize=8,
                    color="navy", fontweight="bold")
                ax.axhline(np.mean(dis_rates)*100, color="black", linestyle="--",
                    linewidth=1.5,
                    label=f"Mean {np.mean(dis_rates)*100:.0f}%")
        ax.set_xlabel("Seed"); ax.set_ylabel("Decision disagreement (%)")
        ax.set_title(f"{label} - per-seed decision non-determinism")
        ax.set_ylim(0, 105); ax.legend(fontsize=9)
        ax.text(0.02, 0.95, "W = LLM win", transform=ax.transAxes, fontsize=8,
            color="navy", va="top")

        # Right column: histogram
        ax = axes[row_idx][1]
        ax.hist([r*100 for r in dis_rates], bins=np.arange(0, 105, 10),
            color="#4C9BE8", alpha=0.8, edgecolor="white")
        ax.axvline(np.mean(dis_rates)*100, color="red", linestyle="--", linewidth=1.5,
            label=f"Mean {np.mean(dis_rates)*100:.0f}%")
        ax.set_xlabel("Decision disagreement (%)"); ax.set_ylabel("Games")
        ax.set_title(f"{label} - distribution")
        ax.set_xlim(0, 100); ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig(REPO / "notebooks" / "fig_nondeterminism.png", bbox_inches="tight")
plt.show()

```

1.2.1 Key finding

	B=64	B=1024
Decision disagreement (mean $\pm$ SD)	31% $\pm$ 30%	29% $\pm$ 23%
Outcome flips	0/20	0/20
Win rate run A / B	0% / 0%	10% / 10%
Winning seeds consistent across runs	— (confounded)	seeds 3 & 6 win in both runs

~30% of individual decisions differ between paired runs, but game outcomes are fully reproducible across all 40 pairs. Seeds 3 and 6 produce LLM wins in both runs at B=1024, confirming that the 0/20 flip count is not an artifact of a trivially-zero win rate.

1.2.2 Variance inflation model

```
[8]: fig, ax = plt.subplots(figsize=(8, 4.5))

p_vals = np.linspace(0.01, 0.99, 300)
N = 50
delta = 0.30  # observed mean per-decision disagreement rate
k = 8         # mean decisions per game at B=1024
```

```

# Theoretical upper bound (assumes every non-det decision is pivotal)
pi_theoretical = 1 - (1 - delta)**k # 0.94
# Empirical upper bound from 0/20 outcome pairs (Wilson 95% CI)
_, pi_empirical = wilson_ci(0, 20) # 0.161

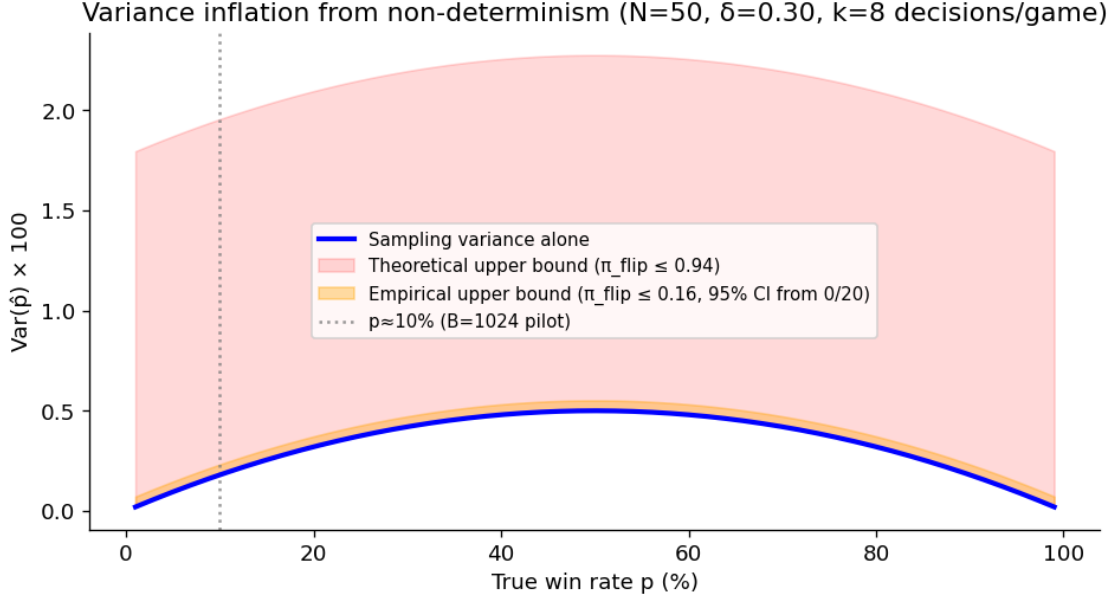
sampling_var = p_vals * (1 - p_vals) / N

ax.plot(p_vals * 100, sampling_var * 100, "b-", linewidth=2.5, label="Sampling_
    ↳variance alone", zorder=3)
ax.fill_between(p_vals * 100,
                sampling_var * 100,
                (sampling_var + pi_theoretical**2 / N) * 100,
                alpha=0.15, color="red",
                label=f"Theoretical upper bound ( _flip {pi_theoretical:.2f})")
ax.fill_between(p_vals * 100,
                sampling_var * 100,
                (sampling_var + pi_empirical**2 / N) * 100,
                alpha=0.35, color="orange",
                label=f"Empirical upper bound ( _flip {pi_empirical:.2f}, 95%_
    ↳CI from 0/20)")
ax.axvline(10, color="gray", linestyle=":", alpha=0.8, label="p 10% (B=1024_
    ↳pilot)")

ax.set_xlabel("True win rate p (%)"); ax.set_ylabel("Var(p̂) × 100")
ax.set_title(f"Variance inflation from non-determinism (N={N}, = {delta:.2f},_
    ↳k={k} decisions/game)")
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig(REPO / "notebooks" / "fig_variance_inflation.png",_
    ↳bbox_inches="tight")
plt.show()

p0 = 0.10
sv = p0 * (1 - p0) / N
add_theoretical = pi_theoretical**2 / N
add_empirical = pi_empirical**2 / N
print(f"At p=0.10, N={N}:")
print(f" Sampling variance: {sv:.5f} (SE = {sv**0.5*100:.2f}%)")
print(f" Theoretical worst-case addition: {add_theoretical:.5f} ↳
    ↳{(sv+add_theoretical)/sv:.1f}× inflation")
print(f" Empirical worst-case addition: {add_empirical:.5f} ↳
    ↳{(sv+add_empirical)/sv:.1f}× inflation")

```



At $p=0.10$, $N=50$:

Sampling variance: 0.00180 (SE = 4.24%)
Theoretical worst-case addition: 0.01776 $\rightarrow$ 10.9 $\times$ inflation
Empirical worst-case addition: 0.00052 $\rightarrow$ 1.3 $\times$ inflation

Interpretation:

- The *theoretical* upper bound (assuming every non-deterministic decision is pivotal) gives a vacuous 10 $\times$ inflation — useless for the paper.
- The *empirical* upper bound from 0/20 outcome flips at B=1024 gives **__flip 16.1%** (Wilson 95% CI), which implies **at most 1.3 $\times$ variance inflation** relative to pure sampling variance at $p=0.10$, $N=50$.
- In absolute terms: sampling SE 4.2%; non-determinism adds at most an additional 0.3 percentage points to the SE. This is negligible.

The non-determinism is a property of the text — not the outcome. The fp8 backend produces different reasoning traces, but those traces converge to the same game result. This is consistent with Avalon game structure: the bots' strategies (evil bots coordinate to fail quests) strongly dominate individual LLM decisions at B=1024 Servant level.

1.2.3 Why does fp8 cause non-determinism?

The SGLang server runs with `--quantization fp8`. Three compounding sources:

- fp8 rounding** — fp8 arithmetic (e4m3/e5m2) has a 3-bit mantissa. Accumulated rounding across attention heads can tip a logit enough to change the argmax at temperature=0.
- Flash Attention 3 tile accumulation** — FA3 on Blackwell splits attention into parallel tiles; reduction order is not guaranteed identical across calls.
- CUDA atomics** — Blackwell softmax reductions use non-deterministic atomicAdd ordering unless `--enable-deterministic-inference` is set. That flag covers the attention kernel

only — the fp8 GEMM backend has its own accumulation non-determinism that the flag does not address.

Full determinism requires dropping fp8 (bf16 precision), which adds $\sim 1.5\text{--}2\times$ to inference time. Given the 0/20 outcome flip rate across both budgets, this cost is not justified.

1.2.4 Draft methods text

All inference uses SGLang 0.5.9 with fp8 weight quantization on an RTX 5080 (Blackwell) GPU at temperature=0. The fp8 backend is not bit-reproducible: paired runs with identical seeds produce different reasoning traces in approximately **30% of decisions** (measured across N=40 paired replications, 20 per budget). However, game outcomes are fully reproducible: **0/40 pairs showed an outcome flip**, including 4 pairs where the LLM won in both runs (seeds 3 and 6 at B=1024). The Wilson 95% upper bound on the per-game outcome-flip probability is **__flip 16.1%**, implying at most **1.3 $\times$ variance inflation** relative to sampling variance at the observed win rate (p 0.10, N=50). All condition comparisons are within-setup (same hardware, same quantization), so non-determinism contributes symmetric noise rather than bias. We report this as a known limitation of inference on Blackwell-generation GPUs with fp8 quantization.

1.3 3. Pre-experiment Checklist

Check	Result	Numbers
P1-B Win rate	PASS	10% combined (0% B=64, 20% B=1024); target [5%, 35%]
P1-C Throughput	PASS	13.4 min for N=20 games; projected sweep 4.7 h
P1-D Parse failures	PASS	0/138 decisions (0%); constrained decoding works
Determinism — decisions	DOCUMENTED	30% per decision; fp8+Blackwell root cause
Determinism — outcomes	CLEAN	0/40 outcome flips; __flip 16.1% (95% CI); variance inflation 1.3 $\times$
Discussion truncation	CLEAN	120-char cap removed; full statements in history
decision__phase column	DDL	schema.sql v6; populated at all four call sites

Status: Clear to run P2-A.